

A Comparative Study of Feature Based, Transformer Based and Prompt Based Methods for Text Classification on 20 Newsgroups Dataset

Abdul Jamil Azizi

Department of Computer Science

Master in IT Digitalization and Sustainability

Lucerne University of Applied Sciences and Arts

Module: Natural Language Processing

Email: abduljamil.azizi@stud.hslu.ch

GitHub: https://github.com/abduljamilazizi/Nalpro_Project

Abstract—This report compares different natural language processing techniques for multi-class text classification on the 20 Newsgroups dataset. The project was divided into four parts. First, a simple neural network with two linear layers and a ReLU activation was trained with different input representations, namely Word2Vec and TF-IDF. An additional experiment was performed with TF-IDF and bigrams while keeping the same network architecture. Second, a BERT-base model was fine-tuned directly for classification. Third, BERT was first adapted with masked language modeling and then fine-tuned for classification. Fourth, Meta-Llama-3-8B-Instruct was evaluated in zero-shot and few-shot settings using prompting. As bonus work, parameter-efficient fine-tuning using QLoRA was also tested under limited GPU resources. The results show that TF-IDF with the simple neural network performed best overall in this project, Word2Vec improved strongly with more training, masked language modeling improved BERT slightly, and few-shot prompting clearly outperformed zero-shot prompting.

Index Terms—Text classification, 20 Newsgroups, TF-IDF, Word2Vec, MLP, BERT, masked language modeling, Llama-3, zero-shot, few-shot, QLoRA

I. INTRODUCTION

Text classification is one of the core tasks in natural language processing and is widely used in topic detection, sentiment analysis, spam filtering, and document organization [7]. The purpose of this project was to compare different text representation and modeling strategies on the same classification task in order to understand their strengths, weaknesses, and practical trade-offs.

Over the past decade, methods for text classification have evolved from classical feature-based approaches such as TF-IDF [6] and Word2Vec [1] to deep contextual models such as BERT [2], and more recently to large language models capable of in-context learning without any gradient updates [3]. Parameter-efficient fine-tuning methods such as LoRA [10] and QLoRA [4] have further expanded the practical options for adapting large models under resource constraints.

The project followed the teacher’s required structure and included four main components:

- 1) a simple neural network with Word2Vec and TF-IDF representations,
- 2) direct fine-tuning of BERT for classification,
- 3) masked language modeling adaptation followed by BERT classification,
- 4) zero-shot and few-shot prompting with Llama-3.

As a bonus, an exploratory experiment with QLoRA-style parameter-efficient fine-tuning was also conducted under Kaggle GPU resource limitations.

The main goal was not only to achieve strong accuracy, but also to compare the effect of input representation, model architecture, adaptation strategy, and prompting method on classification performance.

II. DATASET AND TASK DESCRIPTION

The dataset used in this project was the `fetch_20newsgroups` dataset from scikit-learn [14]. It contains documents from 20 different topic categories and is a well-established benchmark used for multi-class text classification experiments.

The task was to classify each document into one of the 20 topic labels.

The dataset split that was used for this project was:

- Training set: 9,051 samples
- Validation set: 2,263 samples
- Test set: 7,532 samples

The 20 classes of target were: *alt.atheism*, *comp.graphics*, *comp.os.ms-windows.misc*, *comp.sys.ibm.pc.hardware*, *comp.sys.mac.hardware*, *comp.windows.x*, *misc.forsale*, *rec.autos*, *rec.motorcycles*, *rec.sport.baseball*, *rec.sport.hockey*, *sci.crypt*, *sci.electronics*, *sci.med*, *sci.space*, *soc.religion.christian*, *talk.politics.guns*, *talk.politics.mideast*, *talk.politics.misc*, *talk.religion.misc*.

III. PREPROCESSING

Depending on the type of model, preprocessing varied slightly between project components.

A. Preprocessing for Part 1

Stronger preprocessing was used for the basic neural network experiments. The primary actions were:

- lowercasing the text,
- removing punctuation,
- removing digits,
- removing extra spaces,
- tokenizing the cleaned text for Word2Vec.

For TF-IDF, the cleaned text was used directly [6]. For Word2Vec, the cleaned text was converted into token sequences before training the embeddings [1].

B. Preprocessing for Parts 2 and 3

Preprocessing was lighter for the BERT-based experiments because BERT uses its own WordPiece tokenizer [2]. The main preprocessing consisted of:

- using the dataset with headers, footers, and quotes removed,
- tokenizing with the BERT tokenizer,
- truncating or padding to the maximum sequence length.

During the masked language modeling stage in Part 3, an additional token masking procedure was applied following the original BERT pretraining protocol [2].

C. Preprocessing for Part 4

For the Llama prompting experiments, preprocessing was minimal and focused on prompt preparation:

- removing headers, footers, and quotes,
- replacing line breaks,
- normalizing spaces,
- shortening the text to a fixed prompt-friendly length.

IV. EVALUATION METRICS

Three evaluation metrics were used throughout the project:

- **Accuracy**
- **Macro F1**
- **Weighted F1**

Accuracy measures the overall percentage of correct predictions. Macro F1 is especially important in this project because it gives equal importance to all 20 classes, including weaker or less frequent classes [8]. Weighted F1 complements this by taking class frequency into account.

V. PART 1: SIMPLE NEURAL NETWORK WITH WORD2VEC AND TF-IDF

A. Model Architecture

The classifier used in Part 1 was a simple Multi-Layer Perceptron (MLP) [9] with the following structure:

Input Vector $\rightarrow$ Linear Layer $\rightarrow$ ReLU $\rightarrow$ Linear Layer $\rightarrow$ 20 Output Classes

The same architecture was kept fixed across all Part 1 experiments so that the comparison focused on the input representation rather than the classifier design.

B. Word2Vec Experiment

Word2Vec [1] embeddings were trained in two settings:

- short training: 1 epoch
- longer training: 10 epochs

The word embeddings of the words in each document were averaged to produce a single document vector [11]. The MLP classifier was then fed these document vectors.

C. TF-IDF Experiment

In the second Part 1 experiment, the cleaned documents were converted into TF-IDF vectors [6]. TF-IDF is a classical weighting scheme that assigns higher importance to terms that are frequent in a document but rare across the corpus, making it well suited for topic-based classification. The same MLP architecture was then used to process these vectors.

D. Extra Experiment: TF-IDF with Bigrams

As required by the teacher, one additional experiment was performed while keeping the same network unchanged. For this purpose, TF-IDF with bigrams was used to test whether phrase-level information could improve performance compared with unigram TF-IDF [8].

E. Results for Part 1

Table I summarizes the main Part 1 results.

TABLE I
PART 1 RESULTS

Method	Accuracy	Macro F1
Word2Vec + MLP (1 epoch)	0.2262	0.1749
Word2Vec + MLP (10 epochs)	0.5068	0.4804
TF-IDF + MLP	0.6620	0.6530
TF-IDF Bigram + MLP	0.6599	0.6514

F. Important Figures for Part 1

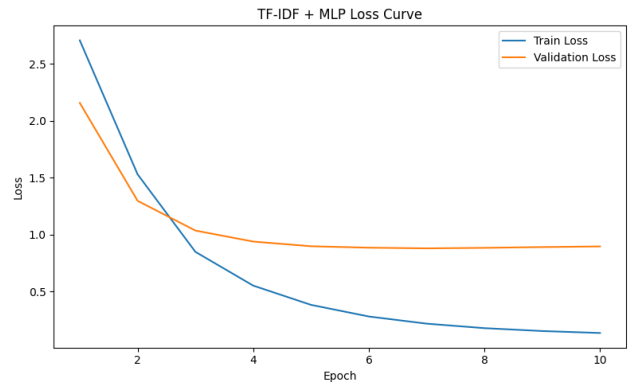


Fig. 1. Training and validation loss for TF-IDF + MLP.

This figure shows the training loss and validation loss of the TF-IDF + MLP model across epochs. It was created to monitor how well the model learned during training and whether it generalized well to unseen validation data. The input to the

model was the TF-IDF vector representation of the cleaned documents, and the output was one of the 20 topic classes. The figure shows that the training loss decreased strongly while the validation loss decreased and then stabilized. This means the model learned useful patterns without severe overfitting. This is a good result because TF-IDF gave the strongest overall performance in Part 1, and the loss behavior was stable compared with the Word2Vec settings.

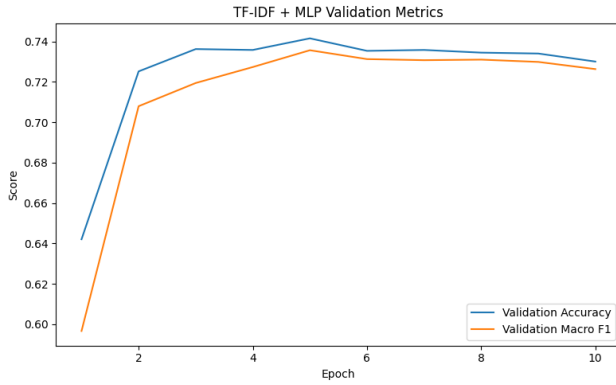


Fig. 2. Validation accuracy and macro F1 for TF-IDF + MLP.

This figure shows the validation accuracy and macro F1 of the TF-IDF + MLP model across epochs. It was used to evaluate how the model performed on unseen validation data during training. The input was the TF-IDF representation of the text, and the output was the predicted topic label. The figure shows that both validation accuracy and macro F1 improved quickly and then remained high. This means the model learned efficiently and performed well across classes. This result is good because high validation accuracy and macro F1 confirm that TF-IDF is a strong representation for this topic classification problem. It also supports the final result table, where TF-IDF outperformed both Word2Vec settings and slightly outperformed the bigram variant.

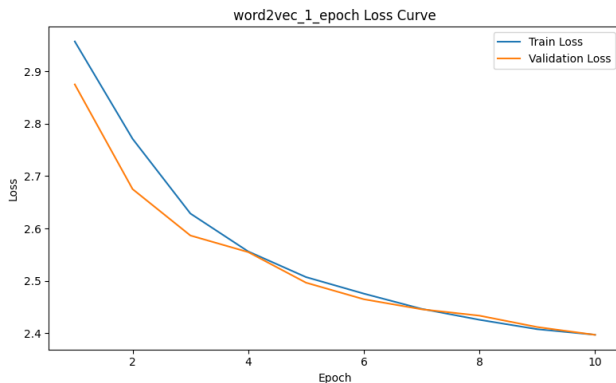


Fig. 3. Training and validation loss for Word2Vec + MLP after 1 epoch of Word2Vec training.

This figure shows the training loss and validation loss when Word2Vec embeddings trained for only 1 epoch were used

as input to the MLP. It was created to examine whether very limited Word2Vec training was enough to produce useful embeddings for classification. The input was the averaged document vector built from Word2Vec word embeddings trained for 1 epoch, and the output was one of the 20 classes. The figure shows that both losses decreased, but they remained high. This means the model learned only weakly because the embeddings were still poor. This result is not strong because one epoch is not enough for Word2Vec to learn rich semantic relationships [1]. Compared with TF-IDF and Word2Vec after 10 epochs, this figure shows much weaker learning behavior.

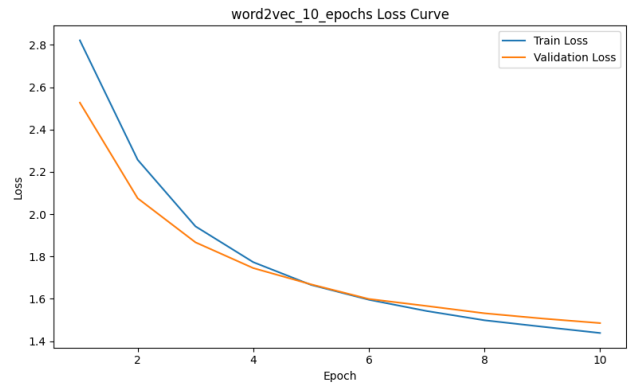


Fig. 4. Training and validation loss for Word2Vec + MLP after 10 epochs of Word2Vec training.

This figure shows the training loss and validation loss when Word2Vec embeddings trained for 10 epochs were used as input to the MLP. It was created to test whether longer Word2Vec training improved classification performance. The input was the averaged document vector built from Word2Vec embeddings trained for 10 epochs, and the output was one of the 20 topic classes. The figure shows that both training and validation loss decreased much more clearly than in the 1-epoch setting. This means the better-trained embeddings helped the classifier learn more effectively. The result is better because longer Word2Vec training produced more meaningful word vectors, which improved document representation [1]. Compared with the 1-epoch Word2Vec figure, this figure clearly shows stronger learning. However, TF-IDF still remained the best overall method.

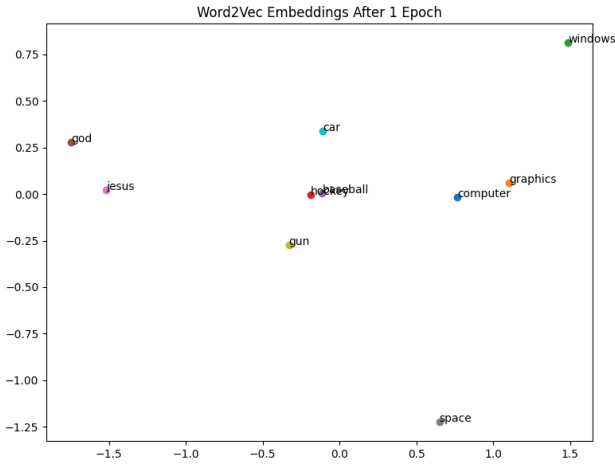


Fig. 5. PCA visualization of Word2Vec embeddings after 1 epoch.

This figure shows a two-dimensional PCA projection of selected Word2Vec word embeddings after 1 epoch of training. It was created to visualize the structure of the embedding space after minimal training. The input was the learned Word2Vec vectors of selected words, and the output was a 2D PCA visualization. The figure shows that the embeddings were still loosely organized and not clearly separated. This suggests that the semantic structure was still weak. The result is weak because one epoch of training was too short for Word2Vec to organize words into meaningful semantic regions. Compared with the 10-epoch PCA figure, this figure is less structured and supports the weaker classification performance of the 1-epoch Word2Vec model.

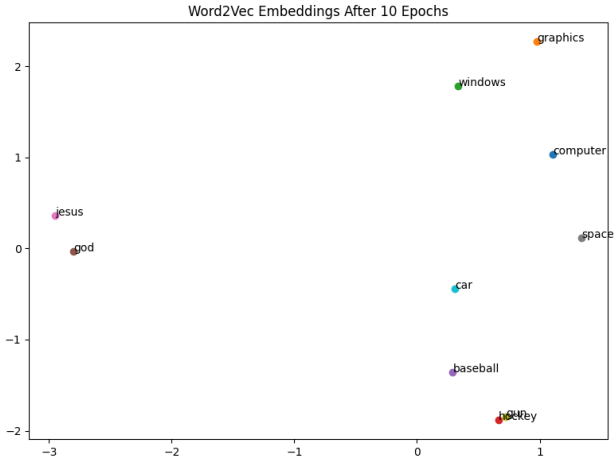


Fig. 6. PCA visualization of Word2Vec embeddings after 10 epochs.

This figure shows a two-dimensional PCA projection of selected Word2Vec word embeddings after 10 epochs of training. It was created to visualize how the embedding space changed after more training. The input was the learned Word2Vec vectors of selected words after 10 epochs, and the output was a 2D PCA visualization. The figure shows that words are more meaningfully separated and grouped. This suggests that

the embedding space became more informative after longer training. The result is better because longer training allowed Word2Vec to learn stronger contextual relationships between words [1]. Compared with the 1-epoch PCA figure, this figure is more structured and supports the improvement seen in the 10-epoch Word2Vec classification result.

G. Discussion of Part 1

The results show that Word2Vec improved substantially when trained for more epochs. The increase from 1 epoch to 10 epochs indicates that embedding quality depends strongly on sufficient training [1]. The PCA visualizations also show that the embedding space became more structured after additional training.

However, TF-IDF still produced the strongest overall result in this project setting. This is likely because the 20 News-groups dataset is strongly topic-based, and TF-IDF captures informative keywords very effectively [6]. The bigram experiment produced a result very close to unigram TF-IDF, suggesting that the added phrase-level information did not significantly improve classification in this case [8].

VI. PART 2: DIRECT BERT FINE-TUNING

A. Method

In Part 2, `bert-base-uncased` was directly fine-tuned on the classification task [2]. BERT uses a bidirectional transformer architecture pretrained on large corpora using masked language modeling and next sentence prediction, making it capable of capturing deep contextual representations [5]. The main settings were:

- maximum sequence length: 64
- batch size: 8
- learning rate: 2×10^{-5}
- epochs: 2

B. Part 2 Experimental Setup

TABLE II
PART 2 TRAINING SETUP

Setting	Value
Model	bert-base-uncased
Maximum sequence length	64
Batch size	8
Learning rate	2×10^{-5}
Epochs	2
Task	20-class classification

C. Results

The direct BERT classification model produced the following result:

TABLE III
PART 2 RESULT

Method	Accuracy	Macro F1
BERT-base Classifier	0.6500	0.6300

D. Discussion

BERT demonstrated the benefit of transformer-based contextual representations [5] and performed well. It was used because it can model the meaning of words in context rather than relying only on isolated keywords [2]. This makes it a strong baseline for modern supervised text classification. However, in this particular project setup, its result was still slightly below TF-IDF + MLP. This suggests that on a strongly topic-based dataset, direct keyword-sensitive representations may still be extremely competitive [7].

VII. PART 3: MASKED LANGUAGE MODELING THEN CLASSIFICATION

A. Method

In Part 3, BERT was first adapted to the dataset using masked language modeling (MLM), where some words were masked and the model learned to predict them [2]. Domain-adaptive pretraining of this kind has been shown to improve downstream task performance by exposing the model to domain-specific vocabulary and style before supervised fine-tuning [12]. After this adaptation stage, the adapted model was fine-tuned for classification.

The MLM stage used:

- MLM subset size: 2,000 samples
- MLM epochs: 1

The downstream classification stage then used the same classification settings as Part 2.

B. Part 3 Experimental Setup

TABLE IV
PART 3 SETUP

Setting	Value
Base model	bert-base-uncased
MLM subset size	2,000
MLM epochs	1
Classification task	20-class classification
Fine-tuning stage	MLM adapted BERT

Table IV summarizes the configuration of the masked language modeling adaptation step. The table shows that the same base BERT model was retained, while an additional intermediate MLM stage was introduced before final classification. The purpose of this setup was to make the model more familiar with the vocabulary and style of the 20 Newsgroups corpus before supervised fine-tuning [12]. This table is important because it clarifies that Part 3 did not use a completely new architecture, but rather an adaptation strategy applied to the same pretrained backbone.

C. Results

The MLM-adapted BERT classifier produced the following result:

TABLE V
PART 3 RESULT

Method	Accuracy	Macro F1
MLM + BERT Classifier	0.6600	0.6390

Table V reports the final performance of the MLM-adapted BERT classifier. The result shows that the model achieved slightly higher accuracy and macro F1 than direct BERT fine-tuning. This indicates that the intermediate adaptation stage helped the model align better with the dataset before solving the classification task [12]. Although the gain is small, it is still meaningful because it demonstrates that adaptation can improve downstream performance even when starting from a strong pretrained transformer [2].

D. Part 2 vs Part 3 Comparison

TABLE VI
DIRECT BERT VS MLM-ADAPTED BERT

Method	Accuracy	Macro F1
Direct BERT	0.6500	0.6300
MLM + BERT	0.6600	0.6390

Table VI provides a direct side-by-side comparison between Parts 2 and 3. This comparison makes the improvement easier to interpret because both rows are based on the same BERT backbone, with the MLM stage as the only major difference. The table shows that the adapted model performed better on both accuracy and macro F1. This supports the argument that domain or corpus adaptation can be useful, even if the final numerical gain is moderate [12].

E. Discussion

The MLM adaptation slightly improved performance over direct BERT fine-tuning. This suggests that adapting the model to the dataset language before classification can be helpful [12]. MLM was used here to expose BERT to the language patterns of the 20 Newsgroups corpus before solving the final supervised classification task [2]. However, the improvement was small because BERT was already strongly pretrained, so the MLM stage acted more as refinement than as a major transformation. Even though the gain was not large, it still shows that domain or dataset adaptation can provide measurable benefit.

VIII. PART 4: LLAMA-3 ZERO-SHOT AND FEW-SHOT PROMPTING

A. Method

In Part 4, `Meta-Llama-3-8B-Instruct` was evaluated in zero-shot and few-shot settings on Kaggle GPU using 4-bit quantization [13]. No fine-tuning or weight updates were performed in this part. Instead, the already pretrained model was guided using prompts, following the in-context learning paradigm introduced in GPT-3 [3].

The setup included:

- model: Meta-Llama-3-8B-Instruct
- platform: Kaggle GPU
- quantization: 4-bit
- test subset: 50 samples

In the zero-shot setting, the model received only the text and the label list. In the few-shot setting, several labeled examples were added to the prompt before the target text.

B. Prompt Design

The zero-shot prompt contained the list of valid labels, the input text, and an instruction to return only one class label. The few-shot prompt used the same structure, but it additionally included several labeled examples before the final unseen text [3]. This allowed the model to learn the task format from the prompt itself during inference, without any gradient-based updates [18].

C. Part 4 Setup

TABLE VII
PART 4 SETUP

Setting	Value
Model	Meta-Llama-3-8B-Instruct
Inference mode	Prompting only
Quantization	4-bit
Test subset	50
Prompt types	Zero-shot and few-shot
Training	None

Table VII summarizes the experimental design for the Llama-based prompting section. It highlights that Part 4 was not a training-based experiment, but an inference-based evaluation using prompts only. This is important because it clearly distinguishes Part 4 from the supervised methods in earlier parts. The table also shows why the comparison with previous parts should be interpreted carefully: Llama-3 was evaluated under a different learning paradigm, where the model relied on prompt instructions and in-context examples instead of gradient-based training [3].

D. Results

The Llama prompting results are shown in Table VIII.

TABLE VIII
PART 4 RESULTS

Method	Accuracy	Macro F1
Llama-3 Zero-Shot	0.1200	0.1040
Llama-3 Few-Shot	0.5000	0.3530

Table VIII shows a large performance gap between zero-shot and few-shot prompting. This result is important because it demonstrates how strongly prompt design influences performance when no training is performed [18]. The few-shot setting clearly helped the model understand the task better by providing labeled examples within the prompt [3]. At the same

time, both results remained below the strongest supervised methods, which shows the limits of prompt-only classification on this dataset.

E. Discussion

Few-shot prompting clearly outperformed zero-shot prompting. This shows that in-context examples helped the model understand both the task and the expected output format [3]. Prompting was used because the project requirement for this part was specifically zero-shot and few-shot evaluation, not fine-tuning. Therefore, the model was not trained in Part 4; instead, it was tested through instruction-based inference [18]. Even though the few-shot result remained below the strongest supervised approaches, it showed that labeled examples inside the prompt can significantly improve performance over pure instruction-only prompting.

IX. PARAMETER-EFFICIENT FINE-TUNING WITH QLoRA

A. Motivation

As bonus work, parameter-efficient fine-tuning was explored to investigate whether adapter-based supervised training could improve over prompt-only methods. The original idea was to fine-tune Llama-3 [13] using LoRA [10] or QLoRA [4]. LoRA reduces the number of trainable parameters by injecting low-rank decomposition matrices into transformer layers [10], while QLoRA further combines this with 4-bit quantization to enable fine-tuning on consumer-grade hardware [4]. However, due to Kaggle GPU memory constraints, the full Llama-3 fine-tuning setup was not feasible in the available environment.

B. Practical Setup

To demonstrate the QLoRA pipeline under limited resources, a smaller instruct model was used. The experiment was carried out on a reduced subset and with lightweight QLoRA settings [4] so that the end-to-end fine-tuning process could be completed successfully.

This experiment’s main point is that it demonstrates that parameter-efficient fine-tuning is possible in the project context, even though it was not carried out at the same scale as the main experiments due to resource limitations.

C. Bonus Setup

TABLE IX
BONUS QLoRA SETUP

Setting	Value
Method	QLoRA
Model type	Smaller instruct model
Training subset	Small reduced subset
Test subset	Small reduced subset
Epochs	1
Environment	Kaggle GPU
Main constraint	Limited memory/resources

D. Results

The QLoRA experiment produced the following result on a small test subset:

TABLE X
BONUS QLoRA RESULT

Method	Accuracy	Macro F1
QLoRA Bonus Experiment	0.2500	0.1750

E. Discussion

This experiment worked technically from start to finish and produced valid predictions. This means that the QLoRA training and evaluation pipeline was implemented successfully [4]. However, its final performance remained below the main project methods. This makes sense given the experiment’s limitations, which included a smaller training subset, a smaller evaluation setup, and limited hardware. Therefore, rather than being a major project conclusion, it is best understood as bonus exploratory work consistent with the proof-of-concept nature of parameter-efficient fine-tuning under constrained conditions [10].

X. FINAL COMPARATIVE ANALYSIS

A. Overall Comparison Table

Table XI summarizes all main experiments and the bonus experiment.

B. Best and Worst Performing Methods

The strongest overall method in this project was TF-IDF + MLP, which achieved an accuracy of 0.6620 and a macro F1 score of 0.6530. This means that, in this setup, the best result did not come from the largest or most modern model, but from a simpler supervised pipeline with a strong feature representation [6]. The weakest main result was Llama-3 zero-shot, with 0.1200 accuracy and 0.1040 macro F1, showing that prompt-only classification without examples was not sufficient for this task [3].

C. Why TF-IDF + MLP Performed Best

TF-IDF + MLP performed best because the 20 Newsgroups dataset is highly topic-oriented [7]. In such a dataset, distinctive keywords often carry strong class information. TF-IDF highlights exactly these important words and downweights common terms [6]. As a result, it creates a very effective input representation for topic classification. The MLP classifier [9] then learns a straightforward mapping from these informative features to the 20 labels.

Another reason for the strong result is that TF-IDF is stable and directly task-relevant in this case. Unlike Word2Vec, it does not first need to learn a dense semantic embedding space [1]. Unlike prompting methods, it does not depend on instruction-following behavior or output formatting [18]. For this reason, TF-IDF produced the strongest and most reliable result in the project.

D. Why Word2Vec Improved with More Training

The Word2Vec results clearly show the importance of embedding quality [1]. With only 1 epoch of Word2Vec training, the embeddings were still weak, which is visible both in the low classification scores and in the PCA visualization of the embedding space. After 10 epochs, the embeddings became more structured and semantically meaningful [1]. This improved the document vectors [11], which then improved classification performance.

However, even after 10 epochs, Word2Vec remained below TF-IDF. A likely reason is that averaged Word2Vec document vectors [11] compress a document into one dense representation, which may lose some class-specific keyword information. TF-IDF, in contrast, preserves strong topic-specific word importance more directly [6].

E. Why the Bigram Experiment Did Not Improve Much

The extra experiment with TF-IDF bigrams was intended to test whether phrase-level information would improve performance while keeping the same network [8]. This was a reasonable idea because some categories may contain informative two-word expressions. However, the final result was almost the same as standard TF-IDF. This suggests that, for this dataset, unigram keyword information was already strong enough, and the added bigram features did not provide much extra benefit. It also suggests that the extra complexity of using bigrams was not strongly rewarded in this classification setting.

F. Why BERT Performed Well but Did Not Win

BERT performed strongly because it uses contextual embeddings based on the transformer architecture [5] and can capture meaning beyond isolated keywords [2]. This makes it generally more flexible and linguistically powerful than feature-based methods. However, in this specific project, BERT still remained slightly below TF-IDF + MLP. One reason is that the task is topic-based rather than deeply semantic in a way that necessarily requires a contextual transformer to outperform all simpler methods [7]. Another reason is that the project used a relatively compact fine-tuning setup, including a short maximum sequence length and a limited number of epochs. Under these conditions, BERT performed very well, but not enough to beat the strongest feature-based baseline.

G. Why MLM + BERT Improved Only Slightly

The MLM-adapted BERT model slightly outperformed direct BERT fine-tuning. This shows that dataset adaptation helped the model adjust to the language and style of the corpus before solving the classification task [12]. However, the improvement was small rather than dramatic. This is expected because BERT was already pretrained on a very large corpus [2] and already started from a strong language understanding base. Therefore, the MLM step acted more as a refinement stage than as a major transformation [12]. The result is still important because it shows that domain or dataset adaptation can provide measurable benefit even if the gain is modest.

TABLE XI
FINAL COMPARISON OF ALL EXPERIMENTS

Experiment	Representation / Strategy	Accuracy	Macro F1	Main Observation
Word2Vec + MLP (1 epoch)	Averaged Word2Vec document vectors with minimal embedding training	0.2262	0.1749	Weak result; embeddings were still immature and not well structured after only one epoch.
Word2Vec + MLP (10 epochs)	Averaged Word2Vec document vectors with longer embedding training	0.5068	0.4804	Strong improvement over 1 epoch; better-trained embeddings improved document representation and classification.
TF-IDF + MLP	Sparse TF-IDF vectors with the same MLP	0.6620	0.6530	Best overall result; topic-sensitive keywords were captured very effectively.
TF-IDF Bigram + MLP	TF-IDF with unigram and bigram features with the same MLP	0.6599	0.6514	Very close to standard TF-IDF; richer phrase features did not produce a meaningful gain.
BERT-base Classifier	Direct transformer fine-tuning for classification	0.6500	0.6300	Strong contextual performance, but still slightly below TF-IDF in this dataset setup.
MLM + BERT Classifier	Dataset adaptation with masked language modeling, then classification fine-tuning	0.6600	0.6390	Slight improvement over direct BERT; adaptation to dataset language helped, but only modestly.
Llama-3 Zero-Shot	Prompting only, no task examples	0.1200	0.1040	Weak result; the model had difficulty mapping the task correctly from instruction alone.
Llama-3 Few-Shot	Prompting with labeled in-context examples	0.5000	0.3530	Much better than zero-shot; examples helped the model understand both task structure and label format.
Bonus QLoRA	Parameter-efficient supervised fine-tuning under constrained resources	0.2500	0.1750	Technically successful but limited by small subset size and restricted hardware; remained below the main methods.

H. Why Few-Shot Prompting Was Better than Zero-Shot

The Llama-3 results show a very large gap between zero-shot and few-shot performance. This happened because the few-shot prompt provided the model with examples of the exact task format: input text followed by the correct label [3]. These examples helped the model infer both the task and the required output style [18]. In zero-shot, the model only received the instruction and the label list, so it had to infer the mapping with no direct examples. That made the task much harder, which explains the very weak result.

Even though few-shot was much better than zero-shot, it still remained below the supervised methods. This is because prompting does not update model weights [3]. It relies only on the pretrained knowledge already inside the model and on the guidance provided by the prompt. In contrast, the supervised methods learn directly from the dataset labels [9].

I. QLoRA Experiment

The bonus QLoRA experiment is important because it demonstrates a working parameter-efficient fine-tuning pipeline [4]. The experiment ran successfully and produced valid predictions, which means the implementation worked technically. However, its final performance remained low compared with the main project methods. The main reason is that the bonus experiment had to be run under strong hardware and memory limitations. Because of this, it used a smaller model, a smaller training subset, and a reduced test setting. Therefore, the QLoRA result should be interpreted as a proof-of-concept or exploratory extension [10] rather than a direct competitor to the main experiments.

J. Deeper Interpretation of the Results

The overall pattern of results shows that stronger model complexity does not automatically guarantee better classification performance [9]. Instead, performance depended strongly on the match between method and task.

For this dataset:

- TF-IDF worked extremely well because topic labels are closely tied to strong keywords [6].
- Word2Vec benefited from more training, which shows the importance of embedding quality [1].
- BERT performed strongly because contextual understanding helps, but the task did not force BERT to dominate simpler methods [2].
- MLM adaptation slightly helped because dataset-specific language adjustment improved the downstream classifier [12].
- Llama few-shot showed that in-context learning can be useful, but prompting alone was still weaker than supervised training [3].
- QLoRA showed that parameter-efficient fine-tuning is feasible, but limited resources strongly constrained the final quality [4].

This comparison also highlights the practical trade-off between simplicity, cost, and performance. The best result came from a relatively simple and efficient model, while the most computationally demanding methods did not always produce the best final accuracy in this project setting.

K. What Could Be Improved in Future Work

Several improvements could be explored in future work to achieve better results:

- train Word2Vec for more epochs and test alternative document aggregation strategies such as Doc2Vec [11] instead of simple averaging,
- tune TF-IDF more extensively, including different vocabulary sizes and n-gram settings [8],
- increase the sequence length and training epochs for BERT [2],
- use a larger MLM adaptation subset and longer MLM training in Part 3 [12],
- improve prompt engineering for Llama-3, including more carefully selected few-shot examples [18],
- evaluate Llama-style models [13] on a larger test subset for more stable comparison,
- scale up the QLoRA experiment with stronger hardware, a larger training set, and a larger instruction-tuned model [4].

A particularly important future improvement would be to run the QLoRA experiment on stronger hardware and with a larger subset, because the current bonus result was limited more by computational constraints than by the method itself [10].

L. Final Comparative Conclusion

The final comparison shows that TF-IDF + MLP was the best-performing method in this project, followed closely by MLM-adapted BERT and direct BERT. Word2Vec improved strongly with more training but remained below TF-IDF. Llama-3 few-shot clearly outperformed zero-shot but did not surpass the supervised methods. The QLoRA bonus experiment successfully demonstrated parameter-efficient fine-tuning, but under limited resources it remained an exploratory extension rather than a top-performing method.

Overall, the project shows that method effectiveness depends not only on model size or modernity, but on how well the method matches the task, the representation, and the available computational budget [9].

XI. LIMITATIONS

This project also has several important limitations. First, some experiments were run under practical time and hardware constraints, especially the Llama-based and QLoRA-based parts [4]. Second, the BERT and MLM experiments were conducted with relatively compact settings rather than a large hyperparameter search [2]. Third, the Part 4 prompting results depended on prompt design and output formatting, which can introduce variability [18]. Fourth, the QLoRA experiment used a much smaller setup than the main experiments, so it can not be interpreted as a fully scaled comparison while there was some resource limitations. [10]. These limitations do not invalidate the results, but they are important when interpreting the final comparisons.

XII. REPOSITORY AND REPRODUCIBILITY

The full project code, documentation, and report repository is available on GitHub:

[Project Repository on GitHub](https://github.com/abduljamilazizi/Nalpro_Project)

https://github.com/abduljamilazizi/Nalpro_Project

This repository contains the implementation of all main project parts, the saved result files, the generated figures, and the supporting project materials used for the final submission. The dataset is loaded automatically via scikit-learn [14] and does not need to be downloaded separately.

XIII. TOOLS AND IMPLEMENTATION NOTES

The project used the following tools and frameworks:

- Python
- scikit-learn [14]
- PyTorch [16]
- Transformers (Hugging Face) [15]
- Gensim [17]
- Kaggle GPU
- AI assistance for presentation wording, explanation support, and organization

AI assistance was used as a support tool only. The experiments, code execution, result interpretation, and final conclusions were based on the project work itself.

XIV. CONCLUSION

This project compared feature-based, transformer-based, and prompt-based approaches on the same multi-class text classification task. The experiments showed that traditional feature-based methods such as TF-IDF [6] remain highly competitive for topic classification, especially when informative keywords are important. Word2Vec [1] improved significantly with more training, showing the importance of embedding quality. BERT [2] performed strongly, and masked language modeling adaptation [12] provided a small but consistent improvement. Llama-3 [13] few-shot prompting [3] was substantially better than zero-shot prompting, but still below the strongest supervised models. Finally, the bonus QLoRA experiment [4] demonstrated that parameter-efficient fine-tuning is feasible in this setting, although its performance remained limited under restricted resources.

Overall, the results show that the best method depends on the task setting, the representation, the model, and the available computation budget.

ACKNOWLEDGMENT

This project was carried out under the supervision of Professor Dr. Forooz Shahbaz Avarvand. The author also acknowledges the requirements of the course and the use of AI as a support tool to draft explanations, improve the wording, and organise the structure of presentation and report. The author has reviewed and agreed to all experiments, interpretation and conclusions.

REFERENCES

- [1] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient Estimation of Word Representations in Vector Space," *arXiv preprint arXiv:1301.3781*, 2013.
- [2] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.

- [3] T. Brown *et al.*, “Language Models are Few-Shot Learners,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 1877–1901.
- [4] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “QLoRA: Efficient Finetuning of Quantized LLMs,” in *Advances in Neural Information Processing Systems*, 2023.
- [5] A. Vaswani *et al.*, “Attention Is All You Need,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 5998–6008.
- [6] K. Sparck Jones, “A Statistical Interpretation of Term Specificity and Its Application in Retrieval,” *Journal of Documentation*, vol. 28, no. 1, pp. 11–21, 1972.
- [7] T. Joachims, “Text Categorization with Support Vector Machines: Learning with Many Relevant Features,” in *Proc. European Conference on Machine Learning (ECML)*, 1998, pp. 137–142.
- [8] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge, UK: Cambridge University Press, 2008.
- [9] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA: MIT Press, 2016.
- [10] E. J. Hu *et al.*, “LoRA: Low-Rank Adaptation of Large Language Models,” in *Proc. International Conference on Learning Representations (ICLR)*, 2022.
- [11] Q. Le and T. Mikolov, “Distributed Representations of Sentences and Documents,” in *Proc. International Conference on Machine Learning (ICML)*, 2014, pp. 1188–1196.
- [12] S. Gururangan *et al.*, “Don’t Stop Pretraining: Adapt Language Models to Domains and Tasks,” in *Proc. Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020, pp. 8342–8360.
- [13] H. Touvron *et al.*, “Llama 2: Open Foundation and Fine-Tuned Chat Models,” *arXiv preprint arXiv:2307.09288*, 2023.
- [14] F. Pedregosa *et al.*, “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [15] T. Wolf *et al.*, “Transformers: State-of-the-Art Natural Language Processing,” in *Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP): System Demonstrations*, 2020, pp. 38–45.
- [16] A. Paszke *et al.*, “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” in *Advances in Neural Information Processing Systems*, vol. 32, 2019, pp. 8024–8035.
- [17] R. Řehůřek and P. Sojka, “Software Framework for Topic Modelling with Large Corpora,” in *Proc. LREC Workshop on New Challenges for NLP Frameworks*, 2010, pp. 45–50.
- [18] P. Liu, W. Yuan, J. Fu, Z. Jiang, H. Hayashi, and G. Neubig, “Pre-train, Prompt, and Predict: A Systematic Survey of Prompting Methods in Natural Language Processing,” *ACM Computing Surveys*, vol. 55, no. 9, pp. 1–35, 2023.